AI 协作日志 W7

文件命名：[姓名][学号]AI 协作日志 W7.docx

姓名：王梓健

学号：2024150200

使用的 AI 工具及方式：Codex（AI 编程助手），先用自然语言描述需求与约束让它生成 V2.0 代码，再由我按结构 / 语法 / 状态流 / 工程惯例四层审查、判定并修改，最后用浏览器实际操作（点击、输入、F12 控制台、刷新、查看 localStorage）加脚本断言验证

Git 仓库链接：https://github.com/kartwatt-cpu/web-week7 （公开仓库，含 feature 分支与合并记录）

分支名：feature/v2-data-driven

任务：完成 V2.0 完整版的数据驱动重构，包括：

1. 补全三个操作函数（addTodo / removeTodo / toggleTodo），遵循“改数据 → 保存 → 重渲染”三步口诀
2. 三个操作都不改原数组——添加用展开、删除用 filter、切换用 map + 展开
3. 筛选与持久化——filter 视图状态切换；localStorage 保存与恢复
4. 完成一次脱离 AI 小改（clearCompleted）——禁用 AI，独立完成

AI 协作记录 1：

类型：（☑ AI 生成 ☐ AI 调试 ☐ 方案比较 ☐ 不用 AI 决策）

对应函数：addTodo / removeTodo / toggleTodo / render（外加 save() / load() / setFilter()）

1. 我的需求（Prompt）

（向 AI 描述需求，要求生成 Todo 列表的 JS 代码。不要贴图。）

我现在的 Todo 列表还是第 6 周的做法：点一次按钮就用 createElement + appendChild 直接改 DOM，状态散在 DOM 里，刷新页面数据就没了。请帮我按「数据驱动」重构成 V2.0：状态收敛到一个 todos 数组，DOM 只由 render() 生成。约束：①三个操作函数 addTodo / removeTodo / toggleTodo 都必须「生成新数组」，不许用 push / splice / 直接改对象属性这类就地修改；②每个操作函数内部固定走「改数据 → save() → render()」三步，不要省中间任何一步；③只能在 render() 里操作 DOM，功能函数里不许

出现 createElement / appendChild / innerHTML / removeChild；④ todos 是唯一的任务状态来源，不要再额外维护一份 UI 状态去跟它同步；⑤视图状态 filter（all / done / active）单独放一个变量，切换筛选只重渲染、不写 localStorage。请给出完整的 JS 代码，并说明每个函数为什么这么写。

2. AI 生成的代码（核心片段）

（粘贴 AI 生成的代码核心部分，不需全部粘贴，但需覆盖你审查的那部分。不要贴图。）

HTML 骨架（只列关键容器，样式沿用第 6 周的主题色与 Flex 布局）：

```
<div class="todo-bar">
  <input type="text" class="todo-input" placeholder="输入待办事项…">
  <button type="button" class="todo-add">添加</button>
</div>
<p class="todo-hint" role="status" aria-live="polite"></p>
<ul class="todo-list" aria-label="待办列表" aria-live="polite"></ul>
<!-- 占位说明：此处按第 6 周约定使用 class 选择器，不再用 id -->
```

JS 骨架（AI 生成，含我审查后要改的地方）：

```
var KEY = 'todos';
var todos = load();
var filter = 'all';

function addTodo(text) {
    todos.push({ id: Date.now(), text: text, completed: false });
    save();
    render();
}
```

```
function removeTodo(id) {
    todos.splice(todos.findIndex(function (t) { return t.id === id; }), 1);
    save();
    render();
}

function toggleTodo(id) {
    for (var i = 0; i < todos.length; i++) {
        if (todos[i].id === id) {
            todos[i].completed = !todos[i].completed;
        }
    }
    save();
    render();
}

function render() {
    list.innerHTML = '';
    var shown = todos.filter(function (t) {
        if (filter === 'all') { return true; }
        if (filter === 'done') { return t.completed; }
        return !t.completed;
    });
    for (var i = 0; i < shown.length; i++) {
        var t = shown[i];
        var li = document.createElement('li');
        li.textContent = t.text;
```

```
            li.dataset.id = t.id;
            li.addEventListener('click', function () { toggleTodo(t.id); });
            var delBtn = document.createElement('button');
            delBtn.textContent = '×';
            delBtn.addEventListener('click', function (e) {
                e.stopPropagation();
                removeTodo(t.id);
            });
            li.appendChild(delBtn);
            list.appendChild(li);
        }
    }
```

3. 我的审查（摘要）

（用一段话概括：你审查了 AI 输出的哪些方面？发现了什么？）

我按四个层次审了 AI 这份骨架。L1 结构与依赖通过：脚本仍放在 body 末尾，用到的容器都已存在；L2 语法与基本用法通过：push / splice / findIndex / forEach / filter 的调用方式都合法，没有语法错误。L3 逻辑与状态流发现三个具体问题：一是三个操作函数全部用了就地修改（push 追加、splice 删除、todos[i].completed 直接改属性），这违反了本周「不改原数组、生成新数组」的工程约束——旧数组被就地改掉后，改之前的状态再也拿不回来，也没法用「新旧状态对比」去排查问题，这属于逻辑与工程层面的问题；二是 render() 里用 var 声明循环变量 i 和 t，而 li 的 click 回调和删除按钮的 click 回调都是闭包，它们捕获的是同一个变量，等用户真的点击时循环早已结束，两个回调拿到的都是最后一条 todo，结果是「点哪一条都变成切换/删除最后一条」；三是 id 用 Date.now() 生成，同一毫秒内连续添加会撞号。L4 工程与惯例发现两点不足：save() 里对 localStorage.setItem 没做异常处理（隐私模式下会直接抛错），load() 里对 JSON.parse 也没做异常处理（存储内容被改坏就会让整页脚本挂掉）。

4. 我的判断和行动

发现错误：直接修改或让 AI 修正 —— 错误一（就地修改）：addTodo 改成 todos = todos.concat([{ id: nextId(), text: text, completed: false }])，removeTodo 改成 todos = todos.filter(function (t) { return t.id !== id; })，toggleTodo 改成 todos = todos.map(function (t) { return t.id === id ? Object.assign({}, t, { completed: !t.completed }) : t; })，三个函数仍是「改数据 → save() → render()」结尾。错误二（var 闭包陷阱）：把 for + var 换成 shown.forEach(function (t) { ... })，每个回调都有自己的 t，li 与删除按钮各自绑定到正确的那条 todo。
发现风险：评估影响范围后决定 —— 风险一（id 撞号）：Date.now() 在同一毫秒内会重复。我没有直接把 AI 的方案照抄，而是加了自增序号：var seq = 0; function nextId() { return Date.now().toString(36) + '-' + (seq++).toString(36); }，影响范围只限 id 生成，结构仍是 id / text / completed。风险二（存储异常）：save() 用 try / catch 包住 setItem，失败时把原因写进提示区；load() 用 try / catch 包住 JSON.parse，并且用 Array.isArray 校验，坏数据退化成空数组而不是让页面白屏。
发现可改进项：主动优化或向 AI 提出要求 —— 复核了可访问性：删除按钮补上 type="button"（避免将来放进表单时触发表单提交）和 aria-label（读屏时能念出删的是哪一条），输入框保留 aria-label，列表与提示区保留 aria-live="polite"；筛选按钮补上 aria-pressed 并在每次 render() 时同步，这样屏幕阅读器能听出当前选中的是哪一个筛选条件。
具体做了什么：

判断类型是「发现错误 + 发现风险」，逐项修改后自己验证，不采用 AI 初稿（判断依据与具体改动见下面三条）。

5. 验证与反思

验证方式：①写完先做静态自查：把页面里的内联脚本单独抽出来跑 node --check 确认没有语法错误，再全局搜一遍 push / splice / innerHTML，确认功能函数里没有就地修改、DOM 操作只出现在 render() 里；②写了一个「DOM 桩 + Node 驱动」的验证脚本，把 index.html 里的脚本原样取出来在桩环境里运行，模拟真实操作：输入内容点「添加」、点条目切换完成状态、点 × 删除、点三个筛选按钮、点「清除已完成」，每一步都同时断言页面元素、localStorage 里的数据和计数行；③在脚本里模拟刷新：把 todos 重新从 localStorage 取一次再渲染，检查数据与完成状态是否恢复、筛选是否回到「全部」；④故意把 localStorage 里的 todos 改成非法 JSON，检查页面是否还能正常打开。一共 37 条断言，覆盖添加 / 空输入 / 切换 / 删除 / 筛选 / 刷新恢复 / 清除已完成 / 坏数据八组场景。
验证结果：node --check 无语法错误；全文搜索确认三个操作函数里没有 push / splice，createElement / appendChild / innerHTML 只出现在 render() 内；37 条断言全部通过，具体看到的是——添加后列表立刻出现新项、textContent 正确、输入框清空、计数行显示「共 n 项 · 未完成 m 项」；纯空格的输入不会添加任何项且提示区出现文字；点条目后该项加上 done 类、localStorage 里对应那条 completed 变成 true，其余条目不受影响；点 × 只移除对应那条，且没有顺带切换完成状态；三个筛选分别只显示全部 / 未完成 / 已完成，按钮的 aria-pressed 跟着同步；重新载入数据后条目数与完成状态与刷新前一致、筛选回到「全部」；把 todos 改成非法 JSON 后页面不抛异常、列表退化为空。

我原来的判断是否得到验证：是。三个判断都被验证了——一是就地修改确实全部消除（三个操作函数里搜不到 push / splice，改完的数组是新对象）；二是 var 循环变量的闭包问题确实存在（改成 forEach 之后，点任意一条的 × 删掉的正是那一条，不再错删最后一条）；三是 Date.now() 撞号确实存在（同一毫秒加两条时旧写法的 id 相同，换成带序号之后两条 id 不同、可以单独删）。

学到的点：①AI 会写出「能跑」的代码，但不一定写出「符合本周工程约束」的代码，像不可变更新这种要求必须写进 Prompt 并逐行检查，否则它会默认用最省事的 push / splice；②审查要盯「状态是怎么变的」，而不只是「页面显示对不对」——把同一段数据在改动前后的两份快照拿出来对比，才能看出是不是就地修改；③循环里创建元素并绑定事件时，先看闭包捕获的是哪一层变量，这一步能提前挡掉「点哪个都删最后一条」这类很难凭肉眼发现的 bug。



图 1　数据驱动重构后的页面（桌面 1000px 视口，截取页头与待办区块）：已完成项显示删除线与灰色文字，底部实时统计“共 3 项 · 未完成 2 项”；截图由无头 Edge 真实渲染，示例数据经 ?demo=1 注入，正常打开页面为空列表



图 2　待办区块特写：输入行、三档筛选按钮、每条右侧的 × 删除按钮、底部计数与“清除已完成”



图 3　筛选“未完成”：已完成的那条不再出现在列表里，而计数行仍按 todos 显示“共 3 项 · 未完成 2 项”——筛选只影响渲染，不改动 todos 本身

补充说明（可选）：

AI 初稿的整体结构（单一数据源、先过滤再渲染、li.dataset.id 定位）是对的，我保留了这个骨架，只重写了三个操作函数、render() 的循环方式、id 生成以及 save() / load() 的异常处理。

AI 协作记录 2：

类型：（☐ AI 生成 ☑ AI 调试 ☐ 方案比较 ☐ 不用 AI 决策）

对应函数：其他（id 生成函数 nextId()，影响 addTodo / removeTodo / toggleTodo 与 render() 里的 li.dataset.id）

1. 我的需求（Prompt）

（向 AI 描述需求，要求生成 Todo 列表的 JS 代码。不要贴图。）

追加一个问题：我按「快速连续添加」的方式测试（大约同一毫秒内添加两条以上），发现两条新任务的 id 一模一样，这时点任意一条的 × 会把两条一起删掉。我的 id 是用 Date.now() 生成的。请在不引入第三方库的前提下给出修复方案，要求 id 仍然保持唯一、并且不改变 todos 里既有数据的结构（还是 id / text / completed 三个字段）。

2. AI 生成的代码（核心片段）

（粘贴 AI 生成的代码核心部分，不需全部粘贴，但需覆盖你审查的那部分。不要贴图。）

AI 修复方案的代码（核心片段，被否决的第一版 + 最终采用版）：

```
// AI 第一版：把时间戳换成随机数（我判断不满足「稳定唯一」，未采用）
function addTodo(text) {
    todos.push({ id: Math.random(), text: text, completed: false });
    save();
    render();
}

// 最终采用：时间戳（36 进制）+ 进程内自增序号，同一毫秒也不会撞号
var seq = 0;
function nextId() {
    return Date.now().toString(36) + '-' + (seq++).toString(36);
```

}

3. 我的审查（摘要）

（用一段话概括：你审查了 AI 输出的哪些方面？发现了什么？）

第二次协作针对一个具体故障。我先复现：连续快速点击「添加」，把任务连加三四条，再从控制台打印 id 列表，看到相邻两条的 id 完全相同；点其中一条的 × 之后两条一起消失。顺着这个现象往回查，问题不在删除逻辑（filter 按 id 过滤是对的），而在 id 生成：Date.now() 只精确到毫秒，真实用户双击或程序化连续调用都会落在同一毫秒，于是新对象的 id 撞号，而「按 id 定位」正是本周删除与切换两条链路的基础，所以撞号的后果是误删、误切换。我把它判定为正确性错误（而不是风格问题），并且确认它只在「同一毫秒连续写」时才出现，属于典型的边界条件缺陷，手点几下不一定能碰出来。

4. 我的判断和行动

发现错误：直接修改或让 AI 修正 —— AI 给的第一个方案是把 Date.now() 换成 Math.random()，我否决了：随机数理论上仍可能重复，而且 id 变成不可读的浮点数，既不好排查也让 localStorage 里的数据更难读。我采用的是「时间戳 + 进程内自增序号」：var seq = 0; function nextId() { return Date.now().toString(36) + '-' + (seq++).toString(36); }，同一毫秒内的第二条会带上序号，不会再撞号。
发现风险：评估影响范围后决定 —— id 结构被改动会影响所有按 id 定位的代码（删除、切换、li.dataset.id），所以我先把这三处调用点逐个过了一遍，确认它们只做「相等比较」、不解析 id 的内容，因此换 id 格式不会波及别的逻辑；同时确认 todos 的数据结构没变，之前存进 localStorage 的旧数据仍然能读出来。
发现可改进项：主动优化或向 AI 提出要求 —— 顺手加了一条断言：连续添加多条后校验 id 集合的大小等于待办条数，把这个边界条件固化进验证脚本，以后再改 id 生成方式会立刻被这条断言拦住。
具体做了什么：

判断类型是「发现错误（正确性缺陷）」：先让 AI 给修复方案，再由我判断取舍、定为最终实现并验证。

5. 验证与反思

验证方式：①先复现：连续快速添加四条任务，把 todos 的 id 列表打印出来逐条比对，确认相邻两条 id 相同、并且点其中一条的 × 会把两条一起删掉；②在验证脚本里加一条针对该边界的断言——同一毫秒内连续添加多条，断言 id 集合的大小等于待办条数，并断言分别点各自的 × 只删掉对应那一条；③以「重新从 localStorage 取数据再渲染」的方式模拟刷新，确认旧数据仍能正常恢复（证明改 id 格式没有破坏已存数据）。

验证结果：修复前该断言失败（两条 id 相同、删一条会少两条），修复后连续添加的 id 全部不同（形如 m3k2p8x1-0、m3k2p8x1-1，前缀相同、序号递增），分别点每条的 × 只删掉对应那一条；重新载入数据后条目与完成状态与刷新前一致，说明旧格式的数据仍能读出来；整份脚本最终 37 条断言全部通过，其中 id 唯一性那条由失败变为通过。
我原来的判断是否得到验证：是。撞号的根因判断（时间戳精度不足）与影响面判断（只波及按 id 比较的逻辑）都成立；部分得到修正的是 AI 的候选方案——Math.random() 看着能解决，实际上不满足「稳定唯一」这个要求，所以最终没有采用它的写法。
学到的点：①故障要先把复现步骤缩到最小（同一毫秒连续加），否则很容易误判成删除逻辑写错了；② AI 的第一版修复常常只是「看起来能治」，要拿需求去衡量它——随机数不保证唯一，就不是合格的 id 方案；③排查问题时先判断「根因在哪一层」再动手，能避免改错地方。
补充说明（可选）：

补充说明（可选）：这次协作的产物是 nextId() 与一条 id 唯一性断言，两者都已经进入最终代码和验证脚本。

记录 3：脱离 AI 小改

5 分钟内禁用 AI 工具，独立完成 clearCompleted() 函数（在 removeTodo 旁边新增，实现“删除所有已完成任务”）。不要求加按钮或绑事件，只要求函数体写对，控制台调用后页面正确更新。

1. 我写的代码

```
// 记录 3：5 分钟内禁用 AI，手写实现「删除所有已完成任务」
// 位置：紧挨 removeTodo() 之后
function clearCompleted() {
  todos = todos.filter(function (t) { return !t.completed; });
  save();
  render();
}
```

2. 我做了什么验证（*怎么验证的？控制台调用后看到了什么？*）



图 4　窄屏（约 490px 视口）下的同一页面：侧栏落到内容下方、技能卡片改为单列，待办区块的输入行与按钮仍不换行溢出——V2.0 的改动没有破坏第 5 周的响应式布局

验证：先在页面上加 4 条任务、点其中 2 条切换成已完成，然后调用 clearCompleted()——因为函数写在 IIFE 里，我是在验证脚本里把它按控制台调用的方式取出来执行的（等价于在 F12 控制台调用）。看到的结果是：列表里 2 条已完成任务立刻消失，剩下 2 条未完成的原样保留，底部计数行同步更新为「共 2 项 · 未完成 2 项」；重新从 localStorage 取数据再渲染一次（等价于刷新页面），这 2 条仍在、已完成的那 2 条没有回来，说明 save() 也生效了；再看 localStorage 里存的内容，剩下的确实只有未完成的 2 条。验证脚本里同样断言了这一步：调用后已完成项全部消失、未完成项条数不变、页面条目数与存储条数一致。

3. 遇到的困难 / 反思（选填） *（禁用 AI 时卡在哪里？怎么解决的？）*

禁用 AI 时卡了一下，卡点在「filter 之后还要不要接着写 save() 和 render()」。写惯了 filter 这种「返回新数组」的方法之后，很容易以为它已经把状态改好了就收工，但本周的规矩是三个操作都必须走完「改数据 → 保存 → 重渲染」三步，少一步的表现就是「控制台里数据对了、页面上还是老样子」或者「页面看着对了、一刷新又回来了」。第二个小犹豫是判断条件：删除已完成应该留下 !t.completed，我一开始写成了 t.completed，在脑子里过了一遍「留下的是未完成的」才改回来。最后的结论是：这一改虽然只有三行，但正好把本周的三步口诀完整用了一次。

记录 4：更多 AI 协作记录（选填）

记录更多本周的 AI 协作过程（比如第 3 个 AI 使用场景、某个特别有价值的调试过程），可以在这里补充。只需一到两句话即可，不必写完整字段。

1. 情境（当时在做什么？）

把页面从 file:// 直接打开换成 python -m http.server 起的本地服务再打开，确认 localStorage 是否仍然正常。

2. 我做了什么 + 结论（一到两句话）

担心数据驱动重构后读取存储会依赖页面来源，于是两种打开方式各验证一遍，结论是同一浏览器下数据都能持久化，未发现问题。

四、5 问说明

以下 5 个问题，检验你是否理解自己提交的代码——用自己的话回答，不要复制 AI 生成的解释。每问一到两句话即可，不必写长。

问题 ① todos 是在哪里更新的？

todos 只在三个操作函数里被重新赋值：addTodo()（用 concat 追加一条新任务）、removeTodo()（用 filter 留下 id 不相等的项）、toggleTodo()（用 map 生成新数组，只把 id 匹配的那一条换成新对象），另外记录 3 手写的 clearCompleted() 里也会重新赋值一次（filter 掉 completed 为 true 的项）。四处都是「用新数组整体替换 todos」，没有一处是在原数组上就地改。

> *要求：指出具体的函数名（至少 3 个）。*

问题 ② render() 如何根据状态生成页面？

render() 是「先筛选、再建元素、最后挂载」：它先清空列表（list.innerHTML = ''），然后按当前 filter 变量用 todos.filter() 算出这一屏该显示哪些任务（这一步不修改 todos 本身），再对筛选结果逐条 createElement 建出 li、写入 textContent 与 dataset.id、给完成项加上 done 类、把点击事件绑到 toggleTodo / removeTodo，最后 appendChild 挂到 ul 上，并同步筛选按钮的 aria-pressed 与底部计数。所以任何一次数据变化之后，只要再调一次 render()，页面就整体重画成新数据的投影，不用手去找要改哪一块 DOM。

> *要求：说明“改数据 → 重渲染”的链路——从 todos 到页面元素，中间经过了什么。*

问题 ③　删除为什么用 filter()？

filter() 会返回一个全新的数组，原数组一个元素都不动，所以「删除前」和「删除后」是两份各自独立的状态；而 splice() 是在原数组上就地摘掉元素，原数组当场被改，改之前的状态就再也拿不回来了。本周要求的「不可变更新」正是要每次修改都产出一个新数组：状态变化看得见（新旧对比就能看出改了哪一条）、排查问题时还能拿旧状态回看。所以删除用 todos = todos.filter(...)，而不是 todos.splice(...)。

> *要求：说清 filter 与“就地修改”的区别。*

问题 ④　刷新后恢复数据依赖哪个函数？

靠 load()。它用 localStorage.getItem('todos') 把存进去的字符串取出来，再做 JSON.parse 变回数组；如果从没存过（返回 null）或者内容被改坏（parse 抛错），它就退化成空数组，让页面正常打开而不是白屏。脚本一开始执行 var todos = load()，所以每次刷新页面都是先把数据取回来，再由 render() 画出来。

> *要求：指出函数名，并简单说一句它做了什么。*

问题 ⑤　指出一处你修改或验证过的 AI 代码

改的是 AI 生成的三个操作函数。以 addTodo 为例，AI 写的是 todos.push({ id: Date.now(), text: text, completed: false });，我改成 todos = todos.concat([{ id: nextId(), text: text, completed: false }]);，removeTodo 由 todos.splice(todos.findIndex(...), 1) 改成 todos = todos.filter(function (t) { return t.id !== id; })，toggleTodo 由在 for 循环里直接写 todos[i].completed = !todos[i].completed 改成 todos = todos.map(function (t) { return t.id === id ? Object.assign({}, t, { completed: !t.completed }) : t; })。改的原因是本周的工程约束要求「不改原数组、生成新数组」，而 push / splice / 直接改属性都是就地修改；改完之后我按添加、切换、删除三条路径逐个操作，并对比刷新前后与 localStorage 里的数据，确认行为不变、状态可追溯。同一份 AI 代码里我还改了 render() 的循环方式（for + var 换成 forEach），因为 var 声明的循环变量被闭包共享，会导致点任意一条 × 都删掉最后一条。

> *要求：给出具体位置（哪个函数/哪几行）+ 改了什么或验证了什么 + 为什么。*

如果本周没有用过 AI：写“本周我自己写的代码，没有用 AI”+ 你验证数据流的方式（如控制台调用 clearCompleted()）。

五、一句话总结

本周你发现 AI 生成的 JS 代码最常出的问题是什么？

AI 生成的 JS 代码最常出的问题是「能跑但不符合本周的工程约束」：一是习惯用 push / splice / 直接改对象属性这类就地修改，把「生成新数组」的不可变更新要求丢在一边；二是状态不收敛，容易同时维护 todos 和一份 UI 状态，改一处忘一处，还常常漏掉 save() 或 render() 里的某一步；三是对边界条件不设防，空输入、同一毫秒撞号的 id、以及 localStorage 读写的异常都没有处理，循环里用 var 声明变量还会埋下闭包共享的隐患。所以我的做法是：Prompt 里就把「不可变更新」「三步口诀」「DOM 只在 render() 里操作」写成硬约束，审查时按结构、语法、状态流、工程惯例四层逐条对照，最后用实际操作加脚本断言去验证，而不是看它跑起来没报错就收货。